StreetCare AI — end-to-end sequence

Every hop of the user story, from the browser to the stores and back — solid: request · dashed: reply · amber: LLM call · green: database write · grey: read

Built from the spec: config/config.py is not in this folder — <names> are config values

Person *browser* · **Dashboard** *Streamlit* · **REST API** *FastAPI · guard* · **tools/ + models** *screening · register · ML* · **LangGraph run** *agentic/graph.py* · **Gemini** *1 call per agent* · **<app database>** *SQLite · 6 tables* · **checkpoint.db** *paused runs* · **chroma_store** *16 PDFs indexed*

STEP 1 — The coordinator signs in and opens SC0052158

- Dashboard → REST API: POST /auth/signin {username, password}
- REST API → <app database>: SELECT user_accounts WHERE username = <coordinator>
- REST API: hash_password(pw, own salt), compared in constant time
- REST API → <app database>: UPDATE user_accounts SET session_token = <SESSION_TOKEN_BYTES> random bytes
- REST API → Dashboard: *200 SignedInUserResponse*
- Dashboard → REST API: GET /queue?defect_type=All
- REST API: authorize_request: token · <PERMISSION_BY_ROUTE>[GET /queue] ∈ role's permissions
- REST API → <app database>: report_status per row: stored review, else <REPORT_STATUS_AWAITING_REVIEW>
- REST API → Dashboard: *200 [QueueRow] — report_queue built once at import (no LLM)*
- Dashboard → REST API: GET /queue/SC0052158
- REST API → Dashboard: *200 ReportDetailResponse — 4 Met, 1 live /0 conflicting, ORDER REPAIR*

STEP 2 — The coordinator opens the review — the run pauses for a Town Centre inspector

- Dashboard → REST API: POST /reviews {report_reference, coordinator_request}
- REST API: 404 unknown · authorize_district · lock · already reviewed → 409
- REST API → LangGraph run: review_workflow.invoke(…, thread SC0052158-<12 hex>)
- LangGraph run → tools/ + models: gather_report_context: profile · documents · register · screening · evidence · ML
- LangGraph run → <app database>: load_report_context: INSERT agent_runs (RUN-<12 hex>, RUNNING)
- LangGraph run → Gemini: plan_report_review → <AGENT_NAME_PLANNER> (1)
- LangGraph run → Gemini: assess_report_requirements → <AGENT_NAME_REQUIREMENTS> (2)
- LangGraph run → Gemini: analyse_report_statements → <AGENT_NAME_EVIDENCE> (3)
- LangGraph run → Gemini: interpret_model_predictions → <AGENT_NAME_RISK> (4)
- LangGraph run → Gemini: prepare_report_recommendation → <AGENT_NAME_WRITER> (5)
- LangGraph run → <app database>: INSERT llm_calls × 5 · INSERT report_reviews (REV-<12 hex>, <REVIEW_STATUS_AWAITING>)
- LangGraph run → checkpoint.db: await_inspector_decision → interrupt(review_id); state saved
- REST API → Dashboard: *201 ReviewRecordResponse — recommended ORDER REPAIR*

STEP 3 — A resident rings Customer Services

- Dashboard → REST API: POST /chat/ask "What is happening with SC-52158?"
- REST API → <app database>: recall_conversation: the adviser's own recent turns
- REST API → chroma_store: search the documents (casebook excluded)
- REST API → Gemini: <AGENT_NAME_STREET_CARE_ASSISTANT> — 1 call → REPORT STATUS, SC0052158
- REST API → <app database>: answer_report_record: the review of the latest revision
- REST API → <app database>: INSERT chat_turns (adviser, REPORT STATUS)
- REST API → Dashboard: *200 <CHAT_ADVISER_STATUS_TEMPLATE>: Footway, Station Road, awaiting a decision*

STEP 4 — A North inspector is refused · the Town Centre inspector orders the repair

- Dashboard → REST API: POST /reviews/REV-<12 hex>/decision (<inspector-n-1>)
- REST API → Dashboard: *403 <DISTRICT_REFUSED_MESSAGE> — before anything is looked at*
- Dashboard → REST API: POST /reviews/REV-<12 hex>/decision {ORDER REPAIR, decision_reason}
- REST API: lock · find_review_conflict, then find_decision_refusal → 409 before resume
- REST API → LangGraph run: resume with {decision, decided_by, decided_by_role, decided_by_district}
- LangGraph run → checkpoint.db: load the paused state
- LangGraph run → tools/ + models: build_works_order_details · standard §7 + §8 guidance
- LangGraph run → Gemini: draft_order_document → <AGENT_NAME_ORDER_WRITER> (6)
- LangGraph run → Gemini: draft_resident_update → <AGENT_NAME_UPDATE_WRITER> (7)
- LangGraph run → <app database>: record_report_outcome (one transaction): INSERT review_decisions
- LangGraph run → <app database>: UPDATE report_reviews → <REVIEW_STATUS_REPAIR_ORDERED> · agent_runs COMPLETE, 7
- REST API → Dashboard: *200 ReviewRecordResponse — works order + resident update, signed <DOCUMENT_SIGN_OFF>*

STEP 5 — The resident calls back · the coordinator audits · signs out

- Dashboard → REST API: POST /chat/ask (adviser) — same report
- REST API → Dashboard: *200 status line + <CHAT_ORDERED_REPAIR_TEMPLATE>: outside no. 17, what the resident was told*
- Dashboard → REST API: POST /chat/ask "Will the hole come back?"
- REST API → Dashboard: *200 <OUTCOME_NOT_PREDICTED_LINE> — and nothing more*
- Dashboard → REST API: GET /reviews/REV-<12 hex>/calls
- REST API → Dashboard: *200 ReviewCallsResponse {call_count 7, expected_call_count 7, Σ tokens, Σ ms}*
- Dashboard → REST API: POST /auth/signout
- REST API → <app database>: UPDATE user_accounts SET session_token = NULL
- REST API → Dashboard: *200 SignedOutResponse {signed_out: true}*

Person · **Dashboard** · **REST API** · **tools/ + models** · **LangGraph run** · **Gemini** · **<app database>** · **checkpoint.db** · **chroma_store**